

With temporal data, the order of the data will impact the model results. Care must be taken to properly encode inputs to the model to handle things such as seasonal effects, or end-of-period (month, quarter, year) impacts. Although XGBoost is not designed specifically for sequential data, it can be adapted to be applied to forecasting-type problems.

Supervised learning is more commonly used for prediction, classification, or regression tasks, while unsupervised learning is more commonly used for exploratory data analysis and discovering hidden patterns or insights in data.

Classification and regression trees (CART) are a type of **supervised learning algorithm** that can be used both for *classification and regression problems*. In a classification problem, the goal is to predict the class, label, or category of a data point or an object. One example of a classification problem is to predict whether there will be customer churn or if a customer will purchase a product based on historical data.

CART models are *built by recursively splitting the data into subsets* based on the value of a feature that best separates the data. The algorithm chooses the feature *that maximizes the separation of the classes or minimizes the variance of the target variable*. The splitting process is repeated until the data are no longer able to be split further.

CART models are easy to explain and can *handle both categorical and numerical features*. However, they can be prone to **overfitting**. **Overfitting** is a phenomenon in machine learning where a model *performs extremely well on the training data but fails to generalize well to unseen data*. Regularization techniques such as **pruning** can be used to **prevent overfitting**. **Pruning** in machine learning *refers to the technique of selectively removing unnecessary or less important features from a model* to improve its efficiency, reduce its complexity, and prevent overfitting.

Advantages of CART models	Disadvantages of CART models
Easy to understand and interpret	Prone to overfitting
Relatively fast to train	Sensitive to noise in the data
Can be used for both classification and regression problems	Can be computationally expensive to train, especially for large datasets, because they need to search through all possible splits in the data in order to find the optimal tree structure

Ensemble modeling is a machine learning technique that combines *multiple models to create a more accurate and robust model*. The *individual models in an ensemble* are called *base models*. The ensemble model *learns from the base models and makes predictions by combining their predictions*.

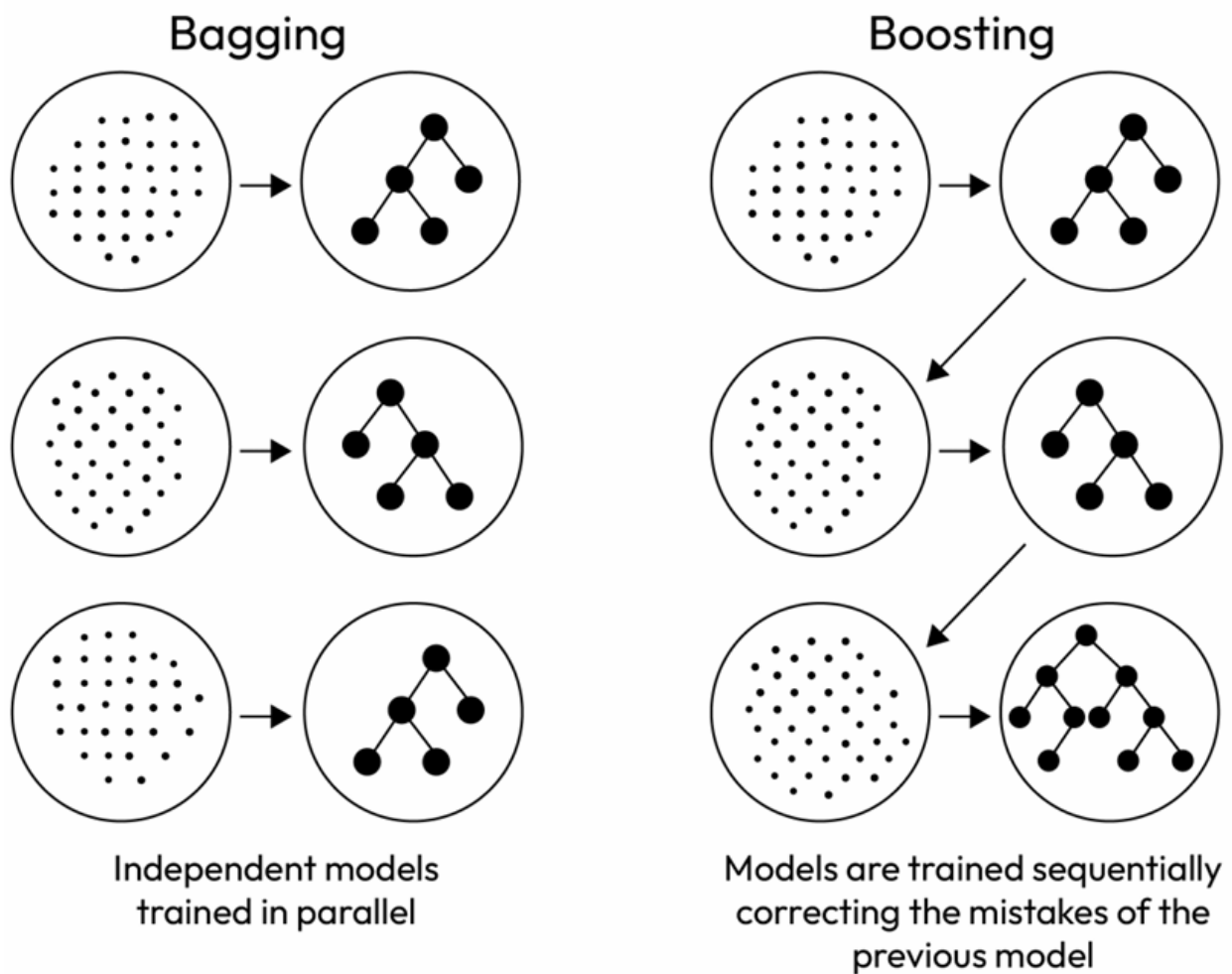
Bagging and *boosting* are *two popular ensemble learning methods* used in machine learning to create more accurate models by combining individual models.

Bagging (*bootstrap aggregation*) creates multiple models *by repeatedly sampling the original dataset* with a *replacement*, which means some *data points may be included in multiple models*, while other data points *may not be included in any models*. Each model is trained on its subset, and *the final prediction is obtained by averaging in the case of regression or voting the predictions of all individual models in the case of classification*. Since it uses a resampling technique, bagging reduces the variance or the impact using a different training set will have on the model.

Boosting is an iterative technique that *focuses on sequentially improving the models*, with each model being *trained to correct the mistakes of the previous models*. To begin with, a base model is trained on the *entire training dataset*. The subsequent models are then *trained by adjusting the weights* to give more importance to the misclassified instances in the previous models. *The final prediction is obtained by combining the predictions of all individual models using a weighted sum*, where the *weights are assigned based on the performance of each model*. Boosting *reduces the bias in the model*. In this context, bias means the assumptions that are being made about the form of the model function. For example, if you use a linear model, you are assuming that the form of the equation that predicts the data is linear – the model is biased towards linear. As you might expect, *decision tree models be less biased than linear regression or logistic regression models*. Boosting iterates on the equation and further reduces the bias.

Bagging	Boosting
Models are trained individually, independently and parallelly	Models are trained sequentially, with each model trying to correct the mistakes of the previous model
Each model has equal weight in the final prediction	Each model's weight in the final prediction depends on its performance

Bagging	Boosting
Variance is reduced and overfitting removed ?	Bias is reduced but overfitting may occur
More accurate ensemble models are created, for example, Random Forest	More accurate ensemble models are created, for example, AdaBoost, Gradient Boosting, and XGBoost



Handling imbalanced data: If there is an imbalance in the data, for example, if there are many more of one category than the others, we can use techniques such as *oversampling* (replicating minority class samples) or *undersampling* (removing majority class samples).

principal component analysis (PCA), which transforms high-dimensional data into lower-dimensional space

Converting categorical data into binary values, such as *one-hot encoding*.

Converting text data into numerical representations, such as *bag-of-words*, and *text embeddings*.

```
98 # make simple report
99 """
100 precision = TP/(TP+FP), recall = TP/(TP+FN), F1score = 2 ((precision * recall) / (precision + recall))
101 """
102 print(classification_report(y_testing_data, t1))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	9
1	0.89	1.00	0.94	8
2	1.00	0.92	0.96	13
accuracy			0.97	30
macro avg	0.96	0.97	0.97	30
weighted avg	0.97	0.97	0.97	30

```

import pandas as pd
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
import xgboost as xgb
import seaborn as sns
import matplotlib.pyplot as plt

data = datasets.load_iris()

data = pd.DataFrame(
    np.c_[data['data'], data['target']], columns=data['feature_names'] +
    ['Species'] # type: ignore
)

data['Species'] = data['Species'].astype(int)

print(data.head())

# Settings the color of palette to Greys
sns.displot(
    data, x='Species', discrete = True, hue='Species', shrink =0.8,
    palette='Greys'
)
plt.show()

fig, axes = plt.subplots(2, 2, figsize=(7, 7))
sns.boxplot(ax=axes[0, 0], data=data, x='Species', y = 'sepal length (cm)',
hue='Species', palette='Greys')
sns.boxplot(ax=axes[0, 1], data=data, x='Species', y = 'petal length (cm)',
hue='Species', palette='Greys')
sns.boxplot(ax=axes[1, 0], data=data, x='Species', y = 'sepal width (cm)',
palette='Greys', hue='Species')
sns.boxplot(ax=axes[1,1], data = data, x='Species', y = 'petal width (cm)',
palette='Greys', hue='Species')
plt.show()

# define x-y plot (pairplot)
sns.set_theme(
    rc={
        "axes.facecolor":"efefef",
        "figure.facecolor":"efefef"
    }
)

```

```

    }
)

graphxy = sns.pairplot(data=data, hue="Species", palette="Greys")
graphxy.add_legend()
plt.show()

# Test the data
train_data, test_data = train_test_split(data, test_size=0.2, random_state=13)
print("The train_data is: ", train_data.shape)
print("The test_data is: ", test_data.shape)

# define the x-y training data for classification
x_training_data = train_data[[
    'sepal length (cm)', 'petal length (cm)',
    'sepal width (cm)', 'petal width (cm)'
]]
print(x_training_data.head())

y_training_data = train_data[['Species']]
print(y_training_data.head())

# define the x-y testing data
x_testing_data = test_data[[
    'sepal length (cm)', 'petal length (cm)',
    'sepal width (cm)', 'petal width (cm)'
]]
y_testing_data = test_data[['Species']]
print(x_testing_data.head())
print(y_testing_data.head())

# train and test the classifier
"""
XGBoost offers multiple evaluation metrics and can accept custom metrics as well.
In addition to
the AUC metric used here, there are over 20 options, including RMSE, mean
absolute percentage
error, and mean average precision.

```

auc: Area under the curve

A default metric is assigned based on whether the model is for prediction (the default is RMSE), classification (the default is logloss), or ranking (the default is map - mean average precision).

```

"""
data_classifier = xgb.XGBClassifier(eval_metric='auc')
data_classifier.fit(
    x_training_data, y_training_data,
    eval_set = [
        (x_testing_data, y_testing_data),
        (x_training_data, y_training_data)
    ]
)

# check the results
t1 = data_classifier.predict(x_testing_data)
print(t1)

# test
t2 = np.array([4.2, 3.1, 1.2, 0.5])
t2 = t2.reshape(1, 4) # model expect array as 2-dimensio
t3 = data_classifier.predict(t2)
print(t3)
# make simple report
"""
precision = TP/(TP+FP), recall = TP/(TP+FN), F1score = 2 ((precision · recall) /
(precision + recall))
"""
print(classification_report(y_testing_data, t1))
# confusion matrix
t4 = confusion_matrix(y_testing_data, t1)
print("Matrix: ", t4)
ConfusionMatrixDisplay.from_predictions(y_testing_data, t1)
plt.show()

*****

```

Gini impurity is a way to measure how well a split has divided the data into classes. It does this by looking at the probability that a random data point from the node is classified incorrectly if randomly assigned a classification based on the distribution of classifications in the node. As the number of classes in the node reduces, the Gini impurity gets closer to zero because there is a lower probability that a random data point will be classified incorrectly. When all the members of a node are in a single class, the Gini impurity is zero.

To handle overfitting, a change the authors made from the standard gradient-boosted tree algorithm is to add a function (Ω , called omega in the paper) for the complexity of the model. This function smooths the weights to avoid overfitting. The omega function does this by penalizing complexity, meaning the algorithm prefers solutions that are simpler. This function also makes the algorithm easier to parallelize for faster computation.

Two additional techniques to handle overfitting are used:

- **Shrinkage**: In this technique, the weights are scaled after each step of the tree boosting.
 - **Subsampling**: Both row subsampling and feature (column) subsampling are used. Subsampling uses a smaller set of features or rows to perform the loss calculation. This is similar to the random forest algorithm where the features are randomly assigned into smaller sets and a tree is built for each.
-

Shrinkage helps with overfitting by reducing the weighting on some parameters, making the model more flexible. Subsampling also works to make a model more flexible by reducing the amount of data used to generate the weights for the model parameters.

The challenge for gradient-boosted tree algorithms is to handle sparse data either caused by missing data or using techniques such as one-hot encoding. One-hot encoding breaks out a column with multiple categories into multiple columns — one for each category — with the rows that belong flagged as 1 and the remaining rows flagged as 0. It creates a lot of rows with zeros in them. Datasets with lots of rows with zeros, missing or non-value data (e.g. NaN), are called sparse data. XGBoost has a sparsity-aware split-finding algorithm where there is a default split direction (left or right) set for each node.

XGBoost uses the exact greedy algorithm to find where to split data, which is the same algorithm that is used in scikit-learn and R's gbm. One challenge to this algorithm is that all data must fit into memory. This is a problem when using the exact greedy algorithm on very large datasets. You need very large amounts of memory to store the entire dataset at once. To address this problem, XGBoost uses an approximation. In this approximation, split candidates are first proposed based on feature distributions (what the histograms of the individual feature columns look like), then the features are mapped into buckets split by those candidates.

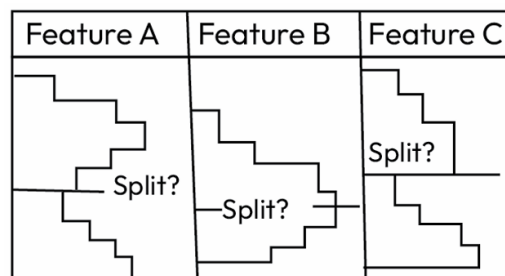


Figure 3.4 – Feature distributions

Finally, the algorithm finds the best split from the proposals based on aggregated statistics. There are two versions of the approximation — one called the global version, which uses the same split candidates from the initial mapping throughout, and one called the local version, which refines the candidates after each split.

The gradient-boosted tree algorithm requires the data to be sorted throughout the processing of the data. Sorting is a compute-intensive task. To improve the overall computational speed, the authors made modifications that optimize for the sort. Data is stored in blocks in a compressed format with each column already sorted by feature value. This helps with parallelization as these blocks can be worked on individually.

The block structure requires a fetch of gradient statistics by row index. If these statistics don't fit into the available memory cache, then there is a delay in the calculations. The approximation algorithm is made aware of the available cache size and based on that the algorithm picks the appropriate size of the data blocks, balancing parallelization and cache size. They also added features to the algorithm to reduce the amount of memory and disk access needed.
